

Automated data science project knowledge graph construction using a document retrieval augmented generation pipeline

Aritha Kumarasinghe, Marite Kirikova

Institute of Applied Computer Systems, Riga Technical University, Riga, Latvia

ARTICLE INFO

Keywords:

Retrieval Augmented Generation
Knowledge Graph
Large Language Model
Data Science

ABSTRACT

The increasing popularity of Large Language Models (LLMs) has driven interest in their integration with knowledge graphs (KGs), particularly for automating KG construction. To explore this approach, we conducted a systematic literature review analyzing existing LLM-based KG construction methods, the resources used, evaluation techniques, and associated challenges. A key challenge identified is the input token constraint of LLMs, which restricts their ability to process extensive documents, such as research articles. To address this challenge, we propose a novel software architecture incorporating a document-based Retrieval-Augmented Generation (RAG) pipeline. The use of this architecture enables LLMs to efficiently extract attributes from external sources, including research papers and project documentation, facilitating the automated construction of a domain-specific KG for data science projects based on user-defined attribute types. The use of the proposed software architecture may support data science researchers and professionals in decision-making and knowledge reuse by providing structured insights from related works. The proposed architecture was tested using 20 real estate analytics projects represented based on 17 attribute types, demonstrating its effectiveness compared to manual knowledge acquisition with an average precision of 75%, recall of 95%, and F1-score of 83%. Additionally, the RAG pipeline reduced input token usage by over 50% compared to full-text processing, significantly lowering computational costs. These findings highlight the potential of RAG-enhanced LLMs for scalable, cost-efficient KG construction.

1. Introduction

A Knowledge Graph (KG) is a knowledge representation tool that allows for the formal representation of real-world entities as triples that are organized based on an underlying ontology that specifies the possible entities, relations, and attributes [1]. One of the main challenges associated with the knowledge graph construction process relates to the acquisition of knowledge from various sources [2]. When using documents, complexities related to document-level entity extraction are caused by cross-sentence and cross-paragraph contexts. One of the potential solutions to this problem is to leverage the context understanding capabilities of pre-trained large language models for entity extraction from contexts to create an LLM-augmented KG [3,4].

In previous work, we proposed a knowledge graph for facilitating knowledge reuse in data analytics constructed by manually defining attributes of completed projects based on the contents of the research articles relating to 20 projects conducted in the domain of real estate analytics [5]. The attributes were organized based on 17 different attribute types, such as Dataset(s) Used, Data Source(s) Used, Data Extraction Techniques Used, and Algorithm(s) Used for Analysis that were defined through a preliminary literature review. In a subsequent work [6] we expanded the scope of research to data science and introduced an ontology to represent data science projects that consist of three main classes: Data Science Project, which represents any project aimed at extracting meaning from data; Data Science Attribute Type, which defines categories of attributes describing different aspects of a project, such as the machine learning algorithm used; and Data Science Project Attribute, which specifies particular attributes within a category, such as Decision Trees, a type of machine learning algorithm. Additionally, we systematically defined 77 data science project attribute types based on the EDISON data science competency framework [7] as instances in a format that is close to natural language such as `DataCollection_Technique_Used`, and `Database_Technology_Used` to ensure that all currently established data science project aspects that relate to a certain resource are represented. This ontology populated with the data science attributes types combined with text from the research article relating to a data analytics project enabled an LLM (GPT 4 by OpenAI) to define RDF triples to represent the research results presented in research articles. This demonstrated the potential of using LLMs with simplified ontologies to reduce the amount of manual effort required to construct a data science project knowledge graph.

This work seeks to expand on this idea by introducing a software architecture that automates the construction of a domain-specific KG for capturing and maintaining detailed knowledge on performed projects in data science based on project documentation or research articles using an LLM. To develop this software architecture, we first conducted a literature review to analyze the current implementations that utilize LLMs for KG construction. The results of the analysis are presented in Section 2, while the amalgamated data that was analyzed, is located in the Appendix using the same numbers for references as in the main body of the article. Building upon the results of the review, in Section 3, we propose an architecture for automating the knowledge graph construction process that uses project documentation/research articles. The results are evaluated by checking the capabilities of the created architecture for recreating the manually created knowledge graph that was proposed in the authors' previous work [5]. The evaluation method and results are presented in Section 4. Section 5 concludes the paper and outlines what future research could be performed.

2. A Literature Review on the use of LLMs in KG construction

This section outlines the methodology used to perform the literature review and introduces the research questions stated to be answered by this literature review (Subsection 2.1). Subsections 2.2-2.4 relate to one or more of the research questions proposed. Subsection 2.2 describes what types of data have been used in the LLM-based KG construction process and identifies a challenge with utilizing the data. Subsection 2.3 provides an overview of the types of LLMs used and what tasks they have carried out in the KG construction process. The final Subsection 2.4 provides insights into evaluation methods/metrics that have been used to evaluate the performance of these models.

2.1. Methodology

A literature review is a tool that can be used to define the state-of-the-art in the domain and to identify the associated challenges [8]. In this work, we accumulate information about the existing software architectures that utilize LLMs for constructing KGs that have been proposed in the literature to understand what and how data sources and technologies (LLMs) are used, and how the proposed architectures are evaluated. Some surveys that consider combining LLMs with KGs have already been published, and we have expanded upon them by emphasizing the architectures proposed for

LLM-constructed knowledge graphs. In the survey [2] regarding automatic knowledge graph construction, pre-trained language models are presented as a very significant solution for automating multiple KG construction tasks, and it provides several examples of the utility of pre-trained LLMs like BERT for these tasks. Another survey [4] explores the combining of LLMs and KGs. It defines three paradigms, one of which is called LLMs augmented KG, where the generalization capabilities of LLMs are used to do tasks within the KG construction process (such as Named Entity Extraction (NER) and Relationship Extraction (RE) from texts). We aim to further investigate how LLMs are used for knowledge graph (KG) construction by conducting a literature review. This review examines studies that propose LLM-based KG construction, focusing on the data sources utilized by LLMs (with an emphasis on language modeling paradigms), the specific KG construction tasks performed, and the methods used to evaluate their performance. Additionally, we compare the performance of LLMs in Named Entity Recognition (NER). The purpose of this review is to guide this work and others who wish to utilize LLMs for KG construction by analyzing previous works that propose such architectures. With this in mind, we defined the following research questions.

RQ1: What software architectures have been proposed that utilize Large Language Models to construct knowledge graphs?

RQ1.1: What sources of data inputs are utilized by the architectures to produce the KG?

RQ1.2: What LLMs have been utilized within the software architectures?

RQ1.3: What tasks have been carried out by the LLMs?

RQ2: What methods and/or metrics are used to evaluate the proposed architectures?

The literature was selected for this review based on inclusion and exclusion criteria represented in Table 1.

Table 1
Inclusion and Exclusion Criteria for the Selection of Research Articles.

Type of Criteria	Criteria No.	Criteria
Inclusion	In1	The peer-reviewed article relates to combining a large language model (LLM) with a knowledge graph.
	In2	The peer-reviewed article relates to utilizing the large language model for constructing a knowledge graph.
Exclusion	Ex1	The peer-reviewed article does not relate to large language models and knowledge graphs.
	Ex2	The full contents of the peer-reviewed article are not accessible.
	Ex4	The peer-reviewed article does relate to large language models and knowledge graphs but does not relate to knowledge graph construction.

To find the relevant literature, we used three scientific databases, namely IEEEExplore, ACM Digital Library, and ScienceDirect using keywords that were defined based on the research questions. The search string consisted of “Large Language Model” AND “Knowledge Graph” AND “Construction”. The additional terms “Constructing” and “LLM” were added with the Boolean search operator “OR” to ensure that variations in the phrasing used within the literature were accounted for resulting in the final search string of:

("Large Language Model" OR "LLM") AND "Knowledge Graph" AND ("Construction" OR "Constructing")

The search was done in the last quarter of 2024 using the advanced search function of each scientific database to ensure the query returns results that are relevant to the research questions. In ScienceDirect, the query was entered into the ‘Title, abstract, or author-specified keywords’ search bar. In IEEEExplore, the command metadata search function was used with the tag used to specify that the search is to be done on the abstract, index terms, and bibliographic citation data (such as document title, publication title, author, etc.). In the ACM Digital Library search query was done on the ACM Full-Text Collection based on the Title, Author Keyword, and Abstract. The query returned 21 results from ScienceDirect, 25 results in IEEEExplore, and 8 results from the ACM Digital Library. To ensure that no articles relating to this topic went unnoticed, we crosschecked the results with title-based querying on Google Scholar. The results were then evaluated based on the inclusion and exclusion criteria (Table 1). Twenty-two papers [9-30] were selected from the results (it should be noted that none of the papers from the ACM Digital Library were selected based on the inclusion and exclusion criteria). The details regarding the selected papers are shown in Fig. 1. Of the selected research articles, 12 were conference proceedings while the remaining 10 were journal articles.

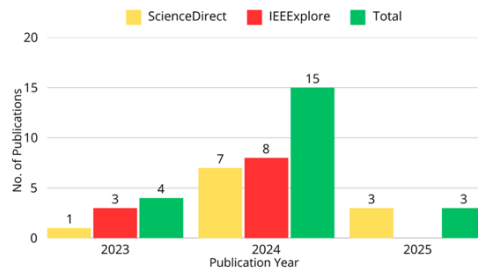


Fig. 1. Statistics for the selected research articles that relate to the construction of knowledge graphs using LLMs.

The interest in the topic is recent, given that the oldest research articles were published in 2023 with the subsequent year of 2024 having more than a threefold increase in the number of publications, with the remaining 3 articles being published as part of journals to be published in 2025.

2.2. RQ1.1: What sources of data inputs are utilized by the architectures to produce the KG?

Given the need for LLMs to use text-based inputs, the sources of data utilized by LLMs for constructing KGs were considered. Table A in the Appendix provides the details concerning the domains in which LLM-constructed KGs have been introduced and what data sources have been utilized to construct knowledge graphs. The overview of data sources used is given in Table 2.

Most of the projects utilize text from documents (Table 2) except for [21], [29], and [30], which use user-generated data such as web-based text data in the form of user reviews [21], and [26] which uses transcribed audio and video data along with documents. Some of the proposed systems are not created to work with a specific type of document (shown in the “general” row in Table 2). Several of the studies used research articles for knowledge graph construction ([12],[16],[18],[24]); two studies utilized only the abstract [18], one article used a combination of the abstracts and full text ([16], [24]), with the remaining one utilizing full text [12]. This article [12], along with two others ([20], [27]), mention limitations imposed on the knowledge graph creation process due to the input token limits that LLMs have.

Table 2
Types of data sources used for LLM-constructed KG.

Type of Document Used	Research Article(s)
Reports	[9], [14], [15], [20]
Research Articles	[12], [16], [18], [24]
Standards/Specifications	[10], [17]
User Generated Data	[21], [29], [30]
General	[11], [19], [27]
Other	[13], [22], [23], [25], [26], [28]

Therefore, the authors of these articles had to implement preprocessing techniques like text segmentation [12] to create text segments that can be inputted into the LLM. This means that the entirety of the documents (such as research articles) must be entered into the LLM as segments to construct the KG. Entering the whole document as segments into the LLM results in an increase in the input tokens used which in turn increases the price of the KG construction process because the usage of LLM APIs (such as the OpenAI’s API) are priced based on the number of input and output tokens used. This limitation imposed by LLMs will be addressed by the software architecture that will be introduced in this work.

2.3. RQ1.2 & RQ1.3: What LLMs have been utilized within the software architectures and what tasks have they carried out?

There are two large language modeling paradigms [31], one of them is the pre-train and fine-tuning paradigm. The pretraining consists of training the large language model on a large corpus to learn universal knowledge and act as a kind of regularization. The finetuning is applied on smaller domain-specific datasets and is used to adjust the parameters of the pre-trained model for a downstream task. An example of an LLM that uses this paradigm is the Bidirectional Encoder Representations from Transformers (BERT) [32] which is a contextualized word representation model that can be pre-trained on a large corpus of text such as large-scale biomedical corpora (PubMed abstracts and PubMed Central full-text articles to create Pretrained Language Models (PLMs) like BioBERT [33]) which is then fine-tuned on various smaller annotated datasets for downstream tasks such as Named Entity Recognition, and Relation Extraction. Another paradigm is called pre-train, prompt, and inference or prompt-based learning, where prompts are inputted into Large Language Models (LLMs) like OpenAI’s GPT-3, GPT-3.5, and GPT-4 models [34]. This paradigm is useful when little data is available for downstream tasks and removes the need for finetuning a pre-trained model, enabling ‘out of the box’ LLMs to perform tasks based on prompts with (few-shot) examples or without (zero-shot) examples. Table B in the Appendix provides information on what pre-trained language models have been used, what types of modeling paradigms have been used in the LLM-augmented Knowledge Graph software architectures, and what tasks have been performed by the PLMs. This information is summarized in Fig. 2 (the colors have the same meaning in both the detailed data in Table B in the Appendix and in Fig. 2).

When looking at the studies that have been conducted (Fig. 2) almost half (48.5%) of the PLMs used were GPT-based PLMs like GPT-4, and GTP 3.5, and these are the most commonly used prompt-based learning PLMs. In two of the projects [9],[18] limitations due to the lack of annotated data for finetuning models like BERT are addressed by automating data annotation using GTP models to create a labeled data set that is then used to fine-tune other models BERT. The tasks carried out by LLMs for the construction of KGs are as follows:

- Triple Extraction: The LLM is tasked with the extraction of triples from a given text that conforms to the Subject, Predicate, Object structure or one that is defined for the specific project.
- Name Entity Extraction: Where a text is entered into the LLM, and the goal is to recognize named entities freely (entity extraction) or based on categories (entity recognition) provided by the user. The most popular type of model used for this task are the GPT-based models.
- Relation Extraction: Where relations between entities are defined by the LLM either based on the text provided (relation extraction) or the entities provided (relation definition).
- Triple verification: Given a text and a set of triples related to the text, the LLM checks whether the triples correspond to the given text. This is done to verify that the generated triples can be traced back to the text provided.
- Other: There were less common (tasks related to only one project) tasks that were carried out by the LLMs such as video or audio-to-text transcription [26], and additional information extraction based on the keywords provided [23].

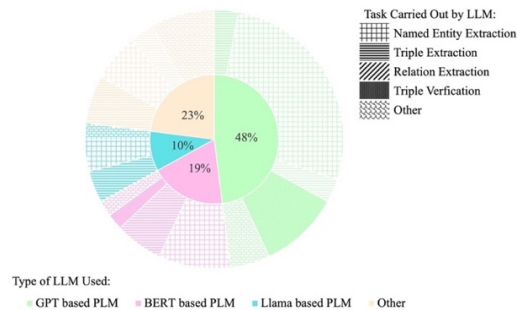


Fig. 2. The proportion of different PLM-based technologies and the tasks that they carried out in previous projects relating to LLM-constructed KGs.

When analyzing the LLMs used and the tasks they have completed, the popularity of GPT-based PLMs like ChatGPT, GPT3.5, and GPT4 specifically for Named Entity Extraction is evident. As mentioned previously, some of the studies [9], [18] used prompt-based learning PLMs to create annotated data for finetuning other models, which were then utilized for KG creation. The advantage of using such an approach is discussed in the next sub-section by comparing the performance results of the different types of LLMs. Further, in Section 3, it is discussed how this approach is combined with Retrieval Augmented Generation (RAG) [35], a technique that allows LLMs to utilize external knowledge from (non-parameterized) knowledge sources (documents, databases, KGs, etc.) to carry out knowledge-intensive NLP tasks without needing to fine-tune a LLM.

2.4. RQ2: What methods and/or metrics are used to evaluate the proposed architectures?

Within the software architectures proposed for the construction of the KGs, PLMs are responsible for completing most if not all of the tasks within the automated knowledge graph construction process as demonstrated in Subsection 2.3. Given that the different studies tested different PLMs for tasks within the KG construction process, Table C in the Appendix presents which LLMs were tested for each task carried out and what evaluation metrics were used to measure their performance. The PLMs highlighted in Table C are the ones that were deemed to perform best because in most studies multiple PLMs or corresponding software architectures are tested to either decide which PLM should be used within the software architecture or to demonstrate how the new software architecture is being proposed performs better than existing solutions.

To compare the performance of the PLMs for Named Entity Recognition tasks we amalgamated the evaluation data for the created models in Table 3. Doing so provides insight into selecting a PLM for the architecture we propose in this work. It should be noted that if multiple datasets are tested in the research study (such as [13]) the mean metric values are presented.

Table 3
Named Entity Recognition Task results for the different models defined in the literature with text in bold specifying the model used with prompt-based learning.

Model	Reference	Precision	Metric Recall	F1-Score
BERT-CRF	[9]	70.98	66.40	68.61
GPT3.5	[9]	83.28	92.64	87.71
GPT4	[9]	92.87	87.26	86.33
LLaMA-15000	[9]	68.72	77.78	72.97
LLaMA-1600	[9]	87.88	83.99	85.89
BioClinicalBERT	[12]	70.09	72.0	71.4
Base GTP4	[12]	43.2	46.3	44.7
AutoRD (utilizing GTP4)	[12]	51.8	61.1	56.1
mBERT-NER	[13]	83.00	78.95	80.92
mRoBERTa-NER	[13]	85.24	58.96	63.60
LS-Llama	[13]	72.21	63.44	67.52
OmEGa (42dot)	[13]	86.60	82.52	84.51
OmEGa (ko-Llama2 7B)	[13]	83.96	79.07	81.44
OmEGa (Llama2 7B)	[13]	72.14	67.52	69.70
BiLSTM-CRF	[15]	73.3	74.4	75.3
BERT-BiLSTM-CRF	[15]	84.0	87.6	85.8
BERT(WWM)-BiLSTM-CRF	[15]	84.1	87.7	85.9
RoBERTa-BiLSTM-CRF	[15]	84.2	87.2	85.8
MBERT-BiLSTM-CRF-SF	[15]	89.9	92.1	91.0
NRFCCKG(BERT)	[17]	-	-	85.67
BERT	[18]	69.39	70.49	69.94
SciBERT	[18]	70.68	71.78	71.23
BioBERT	[18]	73.09	71.99	72.54
MatBERT	[18]	75.78	74.68	75.23
MatSciBERT	[18]	74.26	75.36	74.81
BERT+BiLSTM+CRF	[18]	83.01	82.86	82.94
ChatGPT	[20]	76.0	82.0	78.0
AttackG (ChatGPT)		80.0	47.0	58.0
ChatGLM 6B	[24]	64.80	66.11	65.06
LLaMA2 7B	[24]	72.80	75.35	74.02
BioMedLM	[24]	83.32	82.11	82.69
BERT+BiLSTM+CRF	[29]	93.0	94.0	94.0

When analyzing the performance of Open AI’s out-of-the-box (PLMs without fine-tuning) models like GPT4 in comparison with other models, the Open AI models show good results (75% or more in precision, accuracy, and F1-Score) in two cases ([9] and [20]) and perform poorly (50% or less in precision, accuracy, and F1-Scores) in one of the cases ([12]). However, the poor performance in the case of [12] can be attributed to the out-of-the-box PLM not being trained on sufficient domain (medical) specific data. To compare the overall performance of the prompt-based learning models with the other models tested, we, in Table 4, defined the mean results across the different types of models used for named entity recognition presented in Table 3. The entries with the bold text in the model column of Table 3 specify PLMs used only with prompt-based learning.

Table 4
Mean performance of PLMs used with only prompt-based learning compared to other models.

Paradigm used with the PLM	Mean Accuracy	Mean Precision	Mean F1-Score
Prompt based Learning	72.03	74.86	72.36
Other	77.82	75.63	76.56

There is approximately a 1-6 percent decrease in the performance of using only prompt-based learning models compared with the other models across all the studies considered [9-30]. An aspect that should be taken into consideration when deciding which paradigm to use is the fact that fine-tuning PLMs requires significant computational resources evidenced by the use of 2×3090 GPUs [9] and NVIDIA A100 GPU with 40 GB of memory [13]. Therefore, it can be recommended that out-of-the-box PLMs are better suited for projects that lack computational resources unless a 1-6 percent improvement in performance is required, and given that these models are hosted in external servers, data security, and privacy are also a concern.

3. LLM-Constructed Data Science Project Knowledge Graph

This section relates to the introduction of a novel software architecture that implements a RAG pipeline to facilitate the automated construction of a data science project KG based on project documentation. The first subsection defines the problem addressed by the software architecture. The second subsection will explain how RAG is used in this work. The final subsection defines components of the software architecture, which includes a RAG pipeline.

3.1. Problem Definition

As discussed in the introduction, the data science KG [6] and KG for data analytics [5] constructed in our previous research were intended for reusing research knowledge on related work in data science or analytics. The KG for data analytics [5] was constructed empirically and introduced the idea of presenting completed data analytics projects with the attributes that represent different aspects of a project (e.g., the domain, the data sets/types used, etc.) defined based on a preliminary literature review. The subsequent work expanded the domain of interest to data science and introduced an ontology that consists of classes that represent the attribute and attribute types, which were defined systematically based on the resources identified in the EDISON [7] data science competency framework. The triple structure (Fig. 4) that was used in [5] consists of P, AT, and A where P represents the identifier of the project that has been completed, AT is the type of attribute that is used to represent a certain aspect of a project, such as a project domain, analytics algorithms used, data sets used, machine learning models created, etc., and A represents the attribute of a specific type that represents the project, e.g., Decision Trees can be considered as an analytics algorithm used.



Fig. 4. The triple structure used for representing data science/analytics projects and their aspects.

The definition of P, AT, and A can be done based on the information contained in project documentation or research articles (which was the case in [5]). This activity is resource intensive since documents must be analyzed manually to define attribute(A) of a certain attribute type (AT) that relates to the project(P) and then entered into a data store in tabular format which is used by the software introduced in [5] to generate a KG. The results of using LLMs to automate this work [6] showed a potential for further research. Thus, the goal of this work is to address LLM-based KG construction by proposing a software architecture for automating the KG construction process that takes user-defined attribute types and project or research documents as inputs to construct a KG (utilizing a PLM to define attributes as named entities) and allows users to interact with it. The automated definition of attributes based specifically on research literature comes with unique challenges, one of which relates to the distinction between attributes that represent data science resources (such as analytics algorithms, datasets, software, etc.) that have been used within the project and resources that have been used in other projects which are mentioned in the related work section that a research article can include. The other challenge relates to information that might not be explicitly mentioned in the research article but can be inferred based on the information provided, e.g. an article can mention that a certain database is used but not explicitly mention that the data extraction technique used is database querying. These challenges evolve the attribute definition process from simply extracting or recognizing attributes (named entities) to defining them based on the text provided. These challenges can be addressed by an LLM’s ability to understand contexts and infer missing details; however, a limitation to applying LLMs is the input token size which has been addressed (in [12]) by using methods like splitting documents into segments and then iteratively inputting them into the LLM, but this increases resources that are consumed for the automation.

3.2. Retrieval Augmented Generation Pipeline for Data Science Project Attribute Definition

Retrieval Augmented Generation (RAG) [35] was introduced in 2020 as a technique that allows LLMs to utilize external knowledge from (non-parameterized) knowledge sources (stored in documents, databases, KGs, etc.) to carry out knowledge-intensive Natural Language Processing tasks without needing to fine-tune a PLM. This technique retrieves the relevant data contained in an external data source using a retriever model based on the prompt entered by the user. Then the technique augments the prompt entered into the LLM with the retrieved data. Finally, this data is entered into a PLM model such as BERT to generate the response for the prompt entered by the users. This technique not only allows LLMs to utilize external

knowledge sources to answer user queries but also enables the grounding of responses based on the information within the knowledge sources, thus reducing hallucinations [35].

In this work, the RAG technique is used to address the limitations of LLMs in handling large context data (stored in research articles) due to their input token limits. To implement RAG, we define a pipeline (Fig. 5) that stores the text within research articles as embeddings in an In-Memory Vector Store. Once a user enters an attribute type, relevant text (as chunks) is retrieved by the retriever model based on the semantic similarity between the embeddings that represent the query (which in this case is the attribute type) and the embeddings in the In-Memory Vector Store that represents the text contained with the document/research article. The retrieved text is combined with the attribute type to prompt the LLM to define attributes based on the retrieved text. According to the results of the literature review presented in Section 2, this is the first work to address the issue of the input token limitations of LLMs within the attribute definition process for the construction of KGs based on documentation.

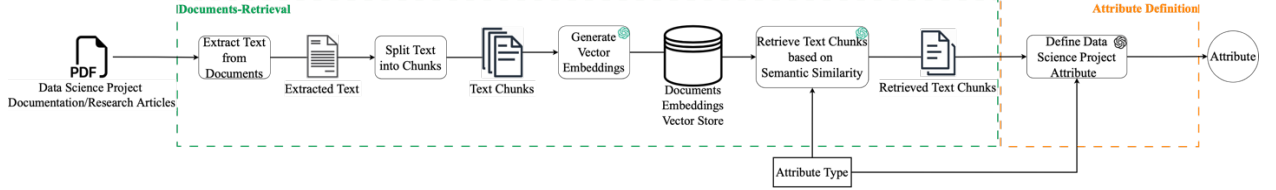


Fig. 5. Document-RAG-based Data Science Attribute Definition Pipeline.

To create a knowledge graph to represent all data science projects the text from each of the research articles must be retrieved so that the LLM can define attributes for each of the projects based on the text contained in the article. To implement this an In-Memory Vector Store that contains the vector embeddings of the text chunks from each document is stored in a dictionary as key-value pairs where the key is the name of the document, and the value is the In-Memory Vector Store that represents the document. This implementation of the RAG pipeline is represented as an algorithm in Algorithm 1.

Algorithm 1 Document-RAG based Attribute Definition Algorithm.

Input:
Dir: Directory that contains Data Science Project documents
Att_Types: Attribute Types that have been entered by the user

Output:
Att: Attributes

Method:
Initialization:
 1: Load **Dir** containing PDF project documents
 2: Initialize the Character Text Splitter
 3: Load Large Language Model
 4: Load Text Embeddings Model
 5: **for** file in **Dir** **do**
 6: Extract text from file
 7: Split the text into chunks using Character Text Splitter
 8: Create In-Memory Vector Store based on the chunks using the Text Embeddings Model
 9: **end for**
 10: **for** type in **Att_Types** **do**
 11: **for** file in In-Memory Vector Store **do**
 12: Retrieve relevant chunks from file based on similarity to type
 13: Define *Att* from the chunks based on an attribute type using the Large Language Model
 14: **return** *Att*
 15: **end for**
 16: **end for**

3.3. Proposed Software Architecture for Automated Data Science Project Knowledge Graph Construction

This work aims to automate the data science KG construction process so that users can explore the knowledge contained within previously completed data science projects. To achieve this goal, we defined a software architecture that implements a document-RAG pipeline that allows an LLM to define attributes to represent completed data science projects based on the contents of project documentation. Additionally, the software architecture implements a web-based user interface similar to the one that was introduced in [5] that allows users to interact with the knowledge graph. Fig. 6 outlines the solution proposed for the document-based construction of the data science KG, which consists of the following functional subsystems defined as functions within a Python program:

- **Documents-Retrieval:** The text contained in each PDF document within the directory specified by the users is extracted using the PyPDFLoader module in the langchain library. This text is then split into chunks (the size of each chunk is at most 1024 characters decided based on the results of [36]) using the CharacterTextSplitter module in the langchain library. These chunks are then converted into text embeddings by OpenAI's text-embedding-ada-002 model (acting as the retrieval model in the RAG pipeline) and are then stored within an In-Memory Vector Store. When a user enters/selects an attribute type via the user interface the same model is then used to retrieve chunks of text that relate to this attribute type based on semantic similarity.
- **Attribute Definition:** Based on the text chunks retrieved and the prompt template (Fig. 7) a zero-shot prompt is inputted into OpenAI's GPT4o model (with the temperature set to 0 to get more deterministic responses) which defines entities that are of the attribute type entered/selected by the user through the user interface for each project based on the text provided (text retrieved by the retrieval model). The GPT is selected in this case based on the results of Section 2.4, which shows that prompt-based learning models (the most commonly used of which are GTP-based models) have comparable performance to other models without requiring annotated data for the fine-tuning process which reduced the amount of computational recourse required for the attribute definition process.
- **Knowledge Graph Storage and Visualization:** The named entities defined by the document-RAG pipeline are then used to create nodes and edges in a Neo4j database. This database is then queried based on the filter options selected by the user to visualize the KG. Instead of using the native Neo4j User Interface, we defined a User Interface (Fig. 8) to make the KG more accessible to users by removing the need to know how to formulate queries. Additionally, to provide the user with information regarding the already established aspects of data science projects, the dropdown menu where the user specifies all aspects of interest (the options are based on the 77 attribute types defined in [6]) while also allowing the user to define their own attribute types.

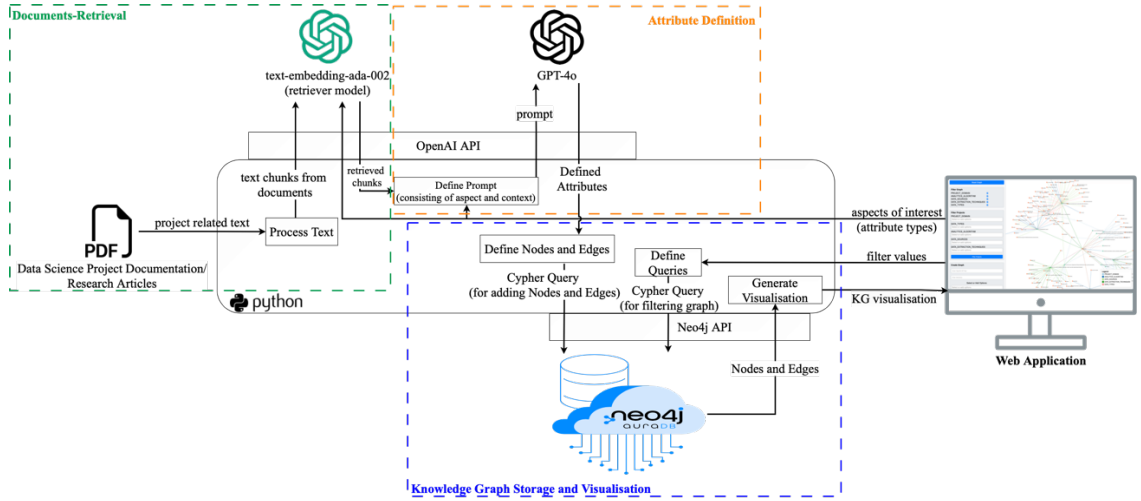


Fig. 6. Proposed software architecture for Document-RAG-based construction of the data science project knowledge graph.

```

Please extract or infer an attribute or attributes(that are concise and have no additional explanations) that represent the project that
relates to the aspect given below based on the information provided below.
If none are identified say "NO".
Return the results as a numbered list with no additional text.
Aspect:
{attribute_type}
Information:
{document_content}

```

Fig. 7. Zero-shot prompt template used to define attributes from the content retrieved from the document (variable: document_content) that are of a certain attribute type (variable: attribute_type).

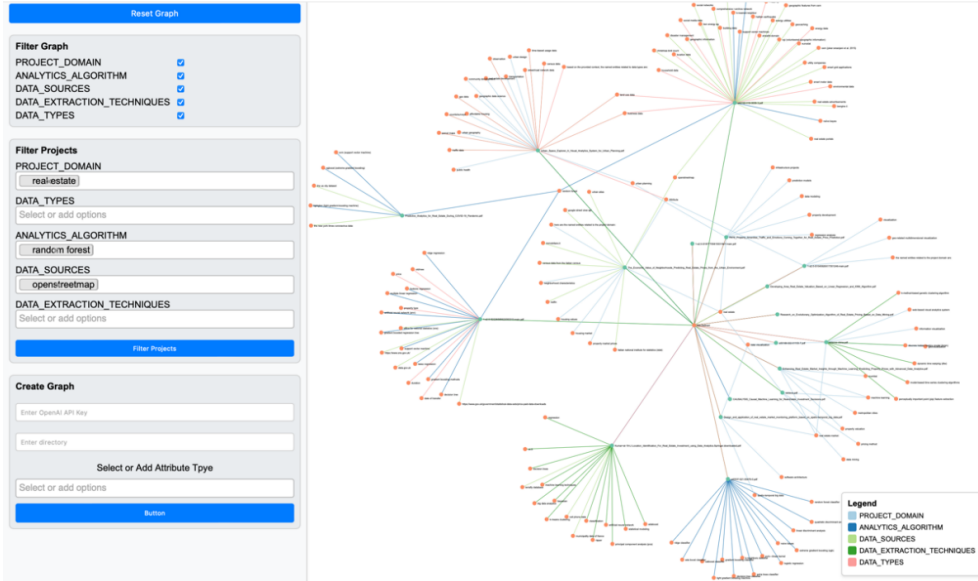


Fig. 8. Web-based user interface for creating and interacting with the data science project knowledge graph.

This proposed software architecture which implements the RAG pipeline to automate the documents-based attribute definition process addresses the input tokens limitation of LLMs within the KG construction process. Additionally, the software architecture includes a user interface that allows users to define any data science attributes of interest and add more documents (research articles, project documentation), facilitating the dynamic construction and expansion of the KG.

4. Testing the Software Architecture

This section relates to the evaluation of the proposed software architecture. Subsection 4.1 outlines the method and metrics used to evaluate the performance of the Document-RAG-based Data Science Attribute Definition Pipeline, which is the subsystem within the software architecture responsible for defining attributes that are then used to construct the data science project KG. Subsection 4.2 shows the results of the evaluation of the model for the attribute definition process. The final section compares the RAG model with using entering the whole text within each document into an LLM.

4.1. Evaluation Method Used

The proposed software architecture is tested by manually comparing the attributes defined by the RAG pipeline with the attributes manually defined to represent 20 real-estate analytics projects (571 attributes of 17 attribute types were identified by reading 20 research articles) [5]. The evaluation metrics used are (1) precision, representing the RAG pipeline’s ability to define only correct attributes from the text; (2) recall, representing the RAG pipeline’s ability to define all the correct attributes presented; and (3) F1-score representing the balance between precision and recall.

$$Precision = \frac{TP}{TP + FP} \times 100\% \quad (1)$$

$$Recall = \frac{TP}{TP + FN} \times 100\%$$

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (3)$$

where TP is the number of correctly identified attributes for a given attribute type, FP stays for incorrectly identified attributes for an attribute type, and FN is the number of attributes that were mentioned in the text but have not been identified.

4.2. Results of Evaluation

When testing the created architecture, the initial results, where the Attribute definition was done based on the attribute type, were unsatisfactory, especially in relation to precision since many of the defined attributes were false positives given that they had no relation to the attribute type. Based on these results, we assumed that the small amount of text (on average three words) used to define the attribute type was inefficient for the retrieval of relevant document chunks and then for the LLM to ‘understand’ the type of attribute that was being requested. To address this issue, we utilized a similar approach to [23] where, based on the attribute type that was entered by the user the LLM was promoted to define what is meant by the attribute type within the context of data science. This output was then used within the document-RAG process and then within the LLM-based attribute definition process. This additional step drastically improved the performance of the software architecture with it being able to achieve an average precision, recall, and F1-score of 75%, 95%, and 83% respectively. The results are shown in Table 5.

Table 5
Results from the evaluation of the Document-RAG based Architecture for Attribute Definition.

Attribute Type	Precision		Recall		F1-Score	
	Attribute Type & Description	Attribute Type	Attribute Type & Description	Attribute Type	Attribute Type & Description	Attribute Type
Project Domain	72.34	71.10	97.14	100	82.93	83.05
Type of Analytics Done	74.29	16.67	100	58.62	85.25	25.95
Project Concerns Object or Measure of Interest	67.78	56.59	100	100	80.79	72.28
Types of Data Used	82.61	43.56	79.17	80.68	80.85	56.57
Dataset Defined	56.78	72.93	100	99.40	71.94	84.13
Data Source Used	94.03	64.95	95.45	92.65	94.74	76.36
Data Extraction Method Used	57.50	4.60	88.46	20.00	69.70	7.48
Data Integration Technique Used	70.00	10.29	100	87.50	82.35	18.42
Analysis Method Used	92.31	14.58	100	58.33	96.00	23.33
Algorithm Used for Analysis	72.92	66.07	100	98.67	84.34	79.14
Models Used for Analysis	70.00	62.16	100	100	82.35	76.67
Software Used for Analysis	64.71	15.22	81.48	53.85	72.13	23.73
Report Defined	62.90	0	95.12	0	75.73	0
Graphical Result Created	82.67	11.51	89.86	25.81	86.11	15.92
Interactive Result Created	95.12	11.59	100	95.00	97.50	20.65
Models Created	81.37	39.13	100	100	89.73	56.25
Results Verified by	77.50	10.71	86.11	15.00	81.58	12.50
Average	74.99	33.63	94.87	69.74	83.18	43.08

4.3. Benefits of Utilizing Document-RAG

During the evaluation process, we utilized the tiktoken library [37] to record the number of tokens needed to represent the contents from the research articles that were retrieved by the retriever model. As the retriever model in the document-RAG pipeline retrieves the relevant document chunks for the definition of attributes, the tiktoken library is used to calculate the number of tokens needed to input the document chunks into GPT-4o. We also used tiktoken to find the number of tokens needed to represent all the text within the research articles used. On average, the number of tokens needed to represent the research articles that relate to the 20 real estate analytics projects was 10485, whereas the document chunks retrieved by the retrieval model could, on average, be represented using 4075 tokens which is a more than a 50% reduction in the tokens needed for the attribute definition process. This reduction in the number of input tokens to the LLM results in more than a 50% reduction in costs incurred for the LLM-based KG construction process.

5. Conclusions

This work introduces a novel software architecture for the automated construction of a data science project knowledge graph based on data science project documentation that implements a document Retrieval Augmented Generation (RAG) pipeline to address the input token limitations that are imposed by LLMs. The knowledge graph represents attributes (based on the aspects of interest of the user such as machine learning algorithms used) of completed data science projects to provide insight for ongoing projects by automatically amalgamating knowledge contained within project documentation. To create the software architecture, we conducted a review of literature that relates to the utilization of Large Language Models (LLMs) for the construction of knowledge graphs (KGs). One of the insights gained from the review was that the use of large documents (such as research papers) in combination with LLMs was hindered by the input tokens limitation that LLMs have. To address this limitation, we utilized the RAG technique implemented as a pipeline within the proposed software architecture to retrieve document chunks based on the aspect of interest defined by the user. This retrieved text is used within a prompt entered into an LLM (GPT-4o) alongside the aspect of interest. The LLM then defines attributes to represent the data science project based on the retrieved text. The proposed software architecture was evaluated by comparing the attributes it defines with those that were manually defined to represent 20 real estate analytics projects with 17 different attribute types (aspects such as project domain, analytics algorithms used, etc.) based on the contents of 20 research articles corresponding to these projects. The results show that document-RAG pipeline was capable of achieving an average precision, recall, and F1-score of 75%, 95%, and 83%, respectively, while utilizing more than 50% fewer input tokens compared to cases where the entirety of the documents were inputted to the LLM demonstrating a high level of performance for the construction of data science project knowledge graph while consuming half the amount of resources (in terms of input token). Additionally, the proposed software architecture includes a web application that allows users to specify aspects of data science projects that they are interested in, as well as allowing the users to interact with the created data science project knowledge graph without having to use complex queries making the knowledge graph more accessible.

In this version of the software architecture, the documents are retrieved from a directory that already contains the documents that have been added by the user, future works can attempt to allow the software architecture to directly access the scientific databases so that the user does not need to manually download research articles to a local directory.

References

- [1] L. Bellomarini, E. Sallinger, S. Vahdati, (2020). Chapter 2 Knowledge Graphs: The Layered Perspective. In: Janev, V., Graux, D., Jabeen, H., Sallinger, E. (eds) Knowledge Graphs and Big Data Processing. Lecture Notes in Computer Science, vol 12072. Springer, Cham. https://doi.org/10.1007/978-3-030-53199-7_2
- [2] L. Zhong, J. Wu, Q. Li, H. Peng, and X. Wu, 2023. A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56(4), pp.1-62.
- [3] P. Shirui, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, "Unifying large language models and knowledge graphs: A roadmap." *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [4] I. Nourhan, S. Aboulela, A. Ibrahim, R. Kashef, "A survey on augmenting knowledge graphs (KGs) with large language models (LLMs): models, evaluation metrics, benchmarks, and challenges." *Discover Artificial Intelligence* 4, no. 1 (2024): 76.
- [5] A. Kumarasinghe, and M. Kirikova, 2024, June. Knowledge Graph for Reusing Research Knowledge on Related Work in Data Analytics. In *International Conference on Advanced Information Systems Engineering* (pp. 186-199). Cham: Springer Nature Switzerland.
- [6] A. Kumarasinghe, and M. Kirikova. (2024). Ontology for data science research results reuse. In *BIR-WS 2024: BIR 2024 Workshops and Doctoral Consortium, 23rd International Conference on Perspectives in Business Informatics Research (BIR 2024)*, September 11-13, 2024, Prague, Czech Rep. [Conference-proceeding]. <https://ceur-ws.org/Vol-3804/paper10m.pdf>
- [7] Y. Demchenko, A. Manieri, A. Belloum, "EDISON Data Science Framework: Part 1. Data Science Competence Framework (CF-DS) Release 2", (2017): URL: https://edison-project.eu/sites/edison-project.eu/files/filefield_paths/edison_cf-ds_release2-v08_0.pdf
- [8] A. Carrera-Rivera, W. Ochoa, F. Larrinaga, G. Lasa, How-to conduct a systematic literature review: A quick guide for computer science research, *MethodsX*, Volume 9, 2022, 101895, ISSN 2215-0161, <https://doi.org/10.1016/j.mex.2022.101895>.
- [9] Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, "LLM-TIKG: Threat intelligence knowledge graph construction utilizing large language model," *Computers and Security*, vol. 145, Oct. 2024, doi: 10.1016/j.cose.2024.103999
- [10] M. Chen, Z. Tao, W. Tang, T. Qin, R. Yang, and C. Zhu, "Enhancing Emergency Decision-making with Knowledge Graphs and Large Language Models," Nov. 2023, doi: 10.1016/j.jidr.2024.104804.
- [11] L. Sun, P. Zhang, F. Gao, Y. An, Z. Li, and Y. Zhao, "SF-GPT: A training-free method to enhance capabilities for knowledge graph construction in LLMs," *Neurocomputing*, vol. 613, Jan. 2025, doi: 10.1016/j.neucom.2024.128726.
- [12] L. Cao, J. Sun, and A. Cross, "An Automatic and End-to-End System for Rare Disease Knowledge Graph Construction Based on Ontology-Enhanced Large Language Models: Development Study," *JMIR Medical Informatics*, vol. 12, pp. e60665-e60665, Dec. 2024, doi: 10.2196/60665.
- [13] M. Shim, H. Choi, H. Koo, K. Um, K. H. Lee, and S. Lee, "OmEGa(Ω): Ontology-based information extraction framework for constructing task-centric knowledge graph from manufacturing documents with large language model," *Advanced Engineering Informatics*, vol. 64, Mar. 2025, doi: 10.1016/j.aei.2024.103001.
- [14] H. Cheng, K. Wang, and X. Tan, "A link prediction method for Chinese financial event knowledge graph based on graph attention networks and convolutional neural networks," *Engineering Applications of Artificial Intelligence*, vol. 138, Dec. 2024, doi: 10.1016/j.engappai.2024.109361.
- [15] D. Liu and L. Cheng, "MAKG: A maritime accident knowledge graph for intelligent accident analysis and management," *Ocean Engineering*, vol. 312, Nov. 2024, doi: 10.1016/j.oceaneng.2024.119280.
- [16] J. Youn, F. Li, G. Simmons, S. Kim, and I. Tagkopoulos, "FoodAtlas: Automated knowledge extraction of food and chemicals from literature," *Computers in Biology and Medicine*, vol. 181, Oct. 2024, doi: 10.1016/j.compbiomed.2024.109072.
- [17] X. Feng et al., "Detecting contradictions from IoT protocol specification documents based on neural generated knowledge graph," *ISA Transactions*, vol. 141, pp. 10-19, Oct. 2023, doi: 10.1016/j.isatra.2023.04.025.
- [18] D. Dai et al., "A GPT-assisted iterative method for extracting domain knowledge from a large volume of literature of electromagnetic wave absorbing materials with limited manually annotated data," *Computational Materials Science*, vol. 246, Jan. 2025, doi: 10.1016/j.commatsci.2024.113431.
- [19] O. Kabal, M. Harzallah, F. Guillet, and R. Ichise, "Enhancing Domain-Independent Knowledge Graph Construction through OpenIE Cleaning and LLMs Validation," in *Procedia Computer Science*, Elsevier B.V., 2024, pp. 2617-2626, doi: 10.1016/j.procs.2024.09.436.
- [20] J. Liu and J. Zhan, "Constructing Knowledge Graph from Cyber Threat Intelligence Using Large Language Model," 2023 *IEEE International Conference on Big Data (BigData)*, Sorrento, Italy, 2023, pp. 516-521, doi: 10.1109/BigData59044.2023.10386611.
- [21] J. Vizcarra, S. Haruta and M. Kurokawa, "Representing the Interaction between Users and Products via LLM-assisted Knowledge Graph Construction," 2024 *IEEE 18th International Conference on Semantic Computing (ICSC)*, Laguna Hills, CA, USA, 2024, pp. 231-232, doi: 10.1109/ICSC59802.2024.00043.
- [22] J. Sun, Z. Zhang, and X. He, "LLM4EduKG: LLM for Automatic Construction of Educational Knowledge Graph," in *Proceedings - 2024 International Conference on Networking and Network Applications*, NaNA 2024, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 269-275, doi: 10.1109/NaNA63151.2024.00051.
- [23] H. Zhao, W. Jiang, J. Deng, Q. Ren and L. Zhang, "Constructing Knowledge Graph for Electricity Keywords Based on Large Language Model," 2023 *IEEE 7th Conference on Energy Internet and Energy System Integration (EI2)*, Hangzhou, China, 2023, pp. 4844-4849, doi: 10.1109/EI259745.2023.10512525.
- [24] L. Ji, S. Du, Y. Qiu, H. Xu and X. Guo, "Constructing a Medical Domain Functional Knowledge Graph with Large Language Models," 2024 *IEEE 6th International Conference on Power, Intelligent Computing and Systems (ICPICS)*, Shenyang, China, 2024, pp. 491-496, doi: 10.1109/ICPICS62053.2024.10796042.

[25]H. Dong, W. Wang, Z. Sun, Z. Kang and X. Ge, "Iterative LLM Prompting for Intelligent Cockpit Knowledge Graph Construction," 2024 4th International Conference on Computer Systems (ICCS), Hangzhou, China, 2024, pp. 136-145, doi: 10.1109/ICCS62594.2024.10795847.

[26]C. S. Lee, M. H. Wang, G. Y. Tseng, C. C. Yue, H. C. Hsieh, and M. Reformat. "Cao Robot for Taiwanese/English Knowledge Graph Application." In 2024 27th Conference of the Oriental COCOSDA International Committee for the Co-ordination and Standardisation of Speech Databases and Assessment Techniques (O-COCOSDA), pp. 1-6. IEEE, 2024.

[27]B. Chen and A. L. Bertozzi, "AutoKG: Efficient Automated Knowledge Graph Generation for Language Models," 2023 IEEE International Conference on Big Data (BigData), Sorrento, Italy, 2023, pp. 3117-3126, doi: 10.1109/BigData59044.2023.10386454.

[28]Y. Fang, Y. Chen, Z. Jiang, J. Xiao and Y. Ge, "Automatic Knowledge Structuration of Automotive User Manual for Question Answering," 2023 4th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+AI), Guiyang, China, 2023, pp. 184-190, doi: 10.1109/ICCBD-AI62252.2023.00038.

[29]S. Rong, R. Li, X. Sun and F. Yi, "Research on Named Entity Recognition Algorithm for Diabetes Intelligent Question and Answer System," 2024 6th International Conference on Internet of Things, Automation and Artificial Intelligence (IoTAAI), Guangzhou, China, 2024, pp. 631-637, doi: 10.1109/IoTAAI62601.2024.10692530.

[30]S. Wen, Y. Chen, X. Pan, W. Zhuang and X. Li, "Enhancing Fault Troubleshooting through Human-Machine Collaboration: A Multi-Stage Reasoning Approach," 2024 IEEE 20th International Conference on Automation Science and Engineering (CASE), Bari, Italy, 2024, pp. 460-467, doi: 10.1109/CASE59546.2024.10711734.

[31]L. Peng, L. Zhang, and J. A. Gulla. "Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems." Transactions of the Association for Computational Linguistics 11 (2023): 1553-1571.

[32]J. Devlin, M.W. Chang, K. Lee, and K. Toutanova. "Bert: Pre-training of deep bidirectional transformers for language understanding." In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pp. 4171-4186. 2019.

[33]J. Lee et al., "BioBERT: A pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, vol. 36, no. 4, pp. 1234–1240, Feb. 2020, doi: 10.1093/bioinformatics/btz682.

[34]K. S. Kalyan, "A survey of GPT-3 family large language models including ChatGPT and GPT-4," Natural Language Processing Journal, vol. 6, p. 100048, Mar. 2024, doi: 10.1016/j.nlp.2023.100048.

[35]P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020, Accessed: Feb. 16, 2025. [Online]. Available: <https://github.com/huggingface/transformers/blob/master/>

[36]K. Juvekar and A. Purwar, "Introducing a new hyper-parameter for RAG: Context Window Utilization," Jul. 2024.

[37]OpenAI and S. Jain., "Tiktoken: Efficient Tokenizer for OpenAI Models," GitHub, <https://github.com/openai/tiktoken>.

Appendix

The reference numbers for the sources in the appendix correspond to the numbers in the list of references of the article.

Table A
Domain in which the LLM-augmented KGs have been applied and Data Sources that are utilized for Knowledge Graph Construction.

Reference	Domain of Utility of the Proposed LLM-Augmented KG Architecture	Data Sources Utilized to Generate the Knowledge Graph
[9]	Threat Intelligence (Cybersecurity)	open-source threat intelligence reports (detailing various malware, vulnerabilities, threat actors, and attack activities)
[10]	Emergency Decision Support	authoritative emergency standards and policy documents
[11]	General	input text X provided within the prompt
[12]	Rare Diseases (Clinical Informatics)	entity and relation types represented within medical ontologies AND medical text include clinical notes, research articles, or any text containing potential rare disease information (the text within the documents was divided into segments less 2000 tokens)
[13]	Manufacturing Industry	classes and relations defined within the Task Centric Ontology and unstructured and domain-specific documents (consisting of text, tables, and images)
[14]	Finance	unstructured Chinese financial research report and entity types and relation types for financial event triplets.
[15]	Maritime	Maritime incident reports
[16]	Nutrition	PubMed abstracts and PMC full-text articles
[17]	Internet of Things	Web protocol specification documents
[18]	Material Science	Defined Entity Categories, and (abstracts from) literature on electromagnetic wave-absorbing materials
[19]	General	Triples and Source Text
[20]	Threat Intelligence (Cybersecurity)	Paragraphs from Cybersecurity Threat Intelligence Reports (60-500 words) relating to technique details, and JSON schema defined based on Ontology.
[21]	User Product Interaction	Raw text data from users' reviews and product catalogs
[22]	Intelligent Education	Unstructured text data encountered in real teaching environments, such as textbooks, tutorials, and course syllabi
[23]	Electric power	Prompt consisting of electric power keyword
[24]	Medical Engineering	Abstracts of medically relevant peer-reviewed research articles from authoritative medical databases as well as their DOIs
[25]	Intelligent Cockpit (Automotive)	Current Interactions defined based on unstructured and semi-structured information, including sensor data, user behavior logs, and external environmental information
[26]	Intelligent Education	Multimodal data, including videos, audio, and text
[27]	General	Discrete text blocks
[28]	User Manual-based Question Answering (Automotive)	Segments of Automotive User Manual
[29]	Diabetes (Clinical Informatics)	Diabetes question and answer data obtained from comprehensive mainstream health websites
[30]	Industrial Machinery Diagnostics	Text from Fault Tickets

Table B

PLMs used within the literature, language modeling paradigm used, and the task carried out by the LLM in the KG-construction process.

Reference	PLM Used	Language Modelling Paradigm Used	Task
[9]	GPT-4	(few-shot) prompt-based learning	Generates fine-tuning dataset consisting of entities and relations
	Llama2-7B	fine-tuning (GPT generated dataset)	Topic classification, named entity recognition and relationship extraction, and TTP classification
[10]	ChatGLM-6b	(few-shot) prompt-based learning	Entities and relations are extracted which are then manually evaluated to construct the knowledge graph
[11]	GPT-4	(few-shot) prompt-based learning	Entities (as well as aliases) and relations are extracted from the text as triples which is done multiple times to address incompleteness or noise in triple
[12]	GPT-4-0613	(few-shot) prompt-based learning	Entities and relations are extracted and then reanalyzed to filter out unrelated entities.
[13]	Ko-Llama2-7B	fine-tuning (manually annotated dataset)	Entities and relations are extracted.
[14]	DeepKE-LLM (with GPT-3.5-turbo as the driving engine)	prompt-based learning	Financial event triples extraction.
[15]	MBERT-BiLSTM-CRF	pretraining (specialized maritime corpora) fine-tuning (using a small set of manually labeled data)	Named Entity Recognition.
[16]	BioBERT	fine-tuning (manually annotated dataset)	Entailment of generated triples(hypothesis) with the text(premise) provided.
[17]	GPT-3.5(text-davinci- 003)	prompt-based learning	Used as a baseline to evaluate the prediction of links between provided entities compared to graph embedding models.
	BERT	pretraining (specification documents of three IoT protocols) train (manually annotated data dataset)	Entity recognition and relation extraction.
[18]	GPT-2-xl	fine-tuning (manually annotated data)	Antecedent, Consequent, and property generation based on rule statements. The generated properties were then related to entities.
	GPT-3.5-turbo	fine-tuning (manually annotated labeled data)	Produced additional annotated labeled data for fine-tuning
	BERT+BiLSTM+CRF	(few-shot) prompt-based learning trained (using manually annotated label data and GPT-produced data)	Extracts entities and relations
[19]	GPT-4	(zero-shot) prompt-based learning	Converts generated triples into the natural language affirmative sentence and then verifies the generated sentence with the original text. Rule-based Noun Phrase Cleaning where the LLM is used to identify the most important word in the subject or object.
[20]	ChatGPT	(zero-shot) prompt-based learning	Extract metadata from text based on JSON schema. Select relevant triples that represent the input text data.
[21]	Not Defined	Not Defined	Annotation of text data relating to user-product interactions.
[22]	GLM4(with P-Tuning v2 fine tuning)	(one shot and chain of thought) prompt-based learning	Extracts entities and relations in the form of triples.
[23]	GPT-4	prompt-based learning	Evaluated the other LLMs generate triples based on the alignment of the triples with the given text chunk.
	ChatGLM3-6B	(zero-shot/few-shot) prompt-based learning	Based on a given keyword the LLM produced related keywords (zero-shot), research field specification (few-shot), and entity concept explanation (few-shot).
	text2vec-large-Chinese	direct input	Reveals the contextual relevance and semantic similarity of electric power keywords.
[24]	BioMedLM	fine-tuning (55 manually annotated medical abstracts)	Named Entity Recognition, Relationship Extract, and Entity Resolution (linking an extracted entity to a unique representation).
[25]	GPT-4-turbo	(zero-shot) prompt-based learning	Extract entities and relationships
[26]	Llama 3.1	(few-shot) prompt-based learning	Extract entities and relations (ontology of terms) based on the input provided.
	OpenAI Whisper model	direct input	Transcribing of videos.
	Sentence BERT	direct input	Comparison of sentence similarity between two input texts.
[27]	gpt-3.5-turbo-16k model	(zero-shot) prompt-based learning	Keyword extraction
[28]	GPT-3.5	(few-shot) prompt-based learning	Extract the subject and predicate (a function that relates the subject to the provided text) based on the text provided.
	GPT-4	(few-shot) prompt-based learning	Evaluated the other LLMs generate triples based on the alignment of the triples with the given text chunk.
[29]	BERT+BiLSTM+CRF	fine-tuning (manually annotated data)	Named Entity Recognition
[30]	GTP-4	(few-shot) prompt-based learning	Named Entity Recognition, Relation Extraction, fault feature classification.

Table C

Evaluation metrics and methods used for the software architectures defined within the literature.

Reference	Evaluated Models/Methods	Tasks	Evaluation Metric
[9]	BERT-CRF, GPT-3.5, GPT-4, LLaMA-15000, LLaMA-1600	Named Entity Recognition	Precision, Recall, F1-Score
	TTPDrill, Tram, LLaMA-Tec , LLaMA-SubTec	TTP classification	Precision, Accuracy, Recall, F1-Score
[10]	Knowledge graph construction is not evaluated		
[11]	GPT-4(ChatIE-like), GPT-4+Entitiy Extraction Filter, SF-GPT	KG triple extraction	Precision, Recall, F1-Score
[12]	AutoRD (GPT-4) , GPT-4, BioClinicalBERT	Entity and Relation Extraction	Precision, Recall, F1-Score
[13]	mBERT-NER, mRoBERTa-NER, OmEGa (42dot) , OmEGa (ko-Llama2) , OmEGa (Llama2)	Instance recognition and relation classification	Precision, Recall, F1-Score
[14]	Knowledge graph construction is not evaluated		
[15]	BiLSTM-CRF BERT-BiLSTM-CRF, BERT(WWM)-BiLSTM-CRF, RoBERTa-BiLSTM-CRF, MBERT-BiLSTM-CRF-SF	Named Entity Recognition	Precision, Recall, F1-Score
[16]	Lit2KG, FoodMine Database	Scope of the Generated Knowledge Graph	Triple quality, Number of chemicals identified for two selected foods.
[17]	NRFCCKG	Named Entity Recognition Condition Split and Property Identification.	Accuracy, F1-Score BLEU Score
[18]	BERT, SciBERT, BioBERT, MatBERT, MatSciBERT, BERT+BiLSTM+CRF	Named Entity Recognition	Precision, Recall, F1-Score
[19]	G_T2KG(Variations) , SciCero, GPT-4	Extracted Triples	Precision, Recall, F1-Score
[20]	ChatGPT , AttackKG ChatGPT , REBEL	Entity Recognition Relation Extraction	Precision, Recall, F1-Score
[21]	Knowledge graph construction is not evaluated		
[22]	ERNIE, Qwen-turbo, GLM4 , ChatGLM2-6B, ChatGLM3-6B, ChatGLM2-6B (P-Tuning v2), ChatGLM3-6B (P-Tuning v2)	Correctness of Extracted Triples	Precision, Recall, F1-Score
[23]	ChatGLM , Llama 2, FreeWilly2	Query Results	ELO Scoring Mechanism
[24]	ChatGLM 6B, LLaMA2 7B, BioMedLM	Named Entity Extraction Relationships Extractions Entity Resolution	Precision, Recall, F1-Score
[25]	GPT-4(variations) , WDec, UIE	Triples Classification	Precision, Recall, F1-Score
[26]	TAIDE, Zephyr, Llama 3.1	Scope of Knowledge Graph	Nodes to Edges Ratio Identified Communities Average betweenness, closeness, eigenvector
[27]	Knowledge graph construction is not evaluated		
[28]	GPT-3.5(with header), GPT-3.5(without header)	Relationship of generated subject and function to the context	Nature Language Inference Score
[29]	BERT+BiLSTM+CRF	Named Entity Recognition	Precision, Recall, F1-Score
[30]	GPT-4	Triple Extraction	Manual Adjustment Cases